

Can better handoffs make an RLM more reliable?

In a recursive language model (RLM), a main model can ask helper models to do smaller tasks. We call this exchange of work and answers a **handoff**.

Last time, we taught the model a basic routine.

Training examples taught it to inspect data and calculate with Python. Asking helpers reliably remained harder.

This time, we studied the handoffs.

Training improved a routine that uses helpers. Separately, we found that linking each helper answer to its input can substantially improve reading accuracy.

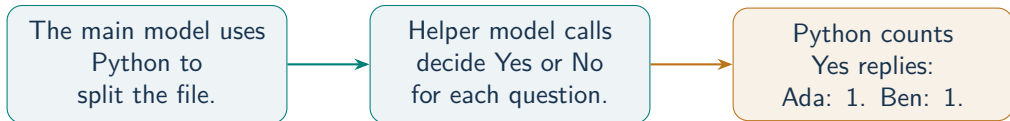
Alex Towell | Advisor discussion | Preliminary findings, 11 September 2026.

An RLM divides a large task into smaller steps.

The file stays in a Python workspace; the main model inspects pieces.

Task: Count each user's questions that ask for a location.

User	Question in the file	Helper: asks for a location?
Ada	Where is Oslo?	Yes
Ada	Who wrote Hamlet?	No
Ben	Where is Kyoto?	Yes



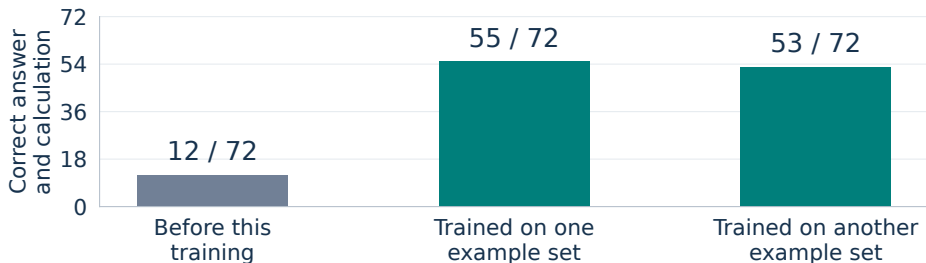
The hope: solve an unfamiliar whole by combining familiar smaller steps.

Invented example of a recursive language model (RLM). Helpers can divide work again; our tests mostly use one helper layer. The main model need not read the whole file at once.

Training taught a more reliable routine for asking helpers and calculating answers.

Worked examples taught the main model to ask helpers to classify text, then count or add in Python.

A success requires the right answer *and* the requested calculation.



Same 72 tasks: new records, familiar task types. Two copies trained on different examples; the helper stayed fixed. Success uses actual helper replies. Eight input sets; missing outcomes allow 12–19, 55, and 53–55 successes. [S3]

We changed how helper answers are linked to the text they describe.

Separate helper test: decide whether a statement follows from a short text.

Example text: *Maya bought a red bike.*

The input carries matching names.

k7ab: Maya bought a vehicle.

z2pm: The bike is blue.

Each answer carries the same name.

k7ab: Supported.

A bike is a vehicle.

z2pm: Contradicted.

The text says red, not blue.

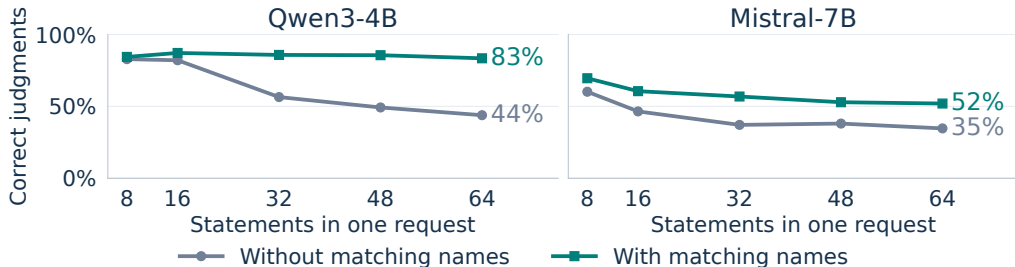
Without these names, the answers are matched only by their position in a list.

The software supplies the names. The model still has to choose the answers.

Illustration, not literal model output. We changed names and formatting, not model weights. The names are arbitrary and give no hint about the correct answer.

Matching names helped helpers judge many statements in one request.

Two language models judged statements about short texts, as in the bike example. Each request asked for several judgments. Neither model was retrained.



Same 16 input sets with and without names; 160 requests per model shown. Three Mistral replies broke the required format and count as wrong. These are helper judgments, not complete RLM solutions. [H8, H9]

Smaller requests were faster than large requests with matching names in this local test.

Each method judged the same 768 statements about short texts. We changed how many statements were sent to the helper together.

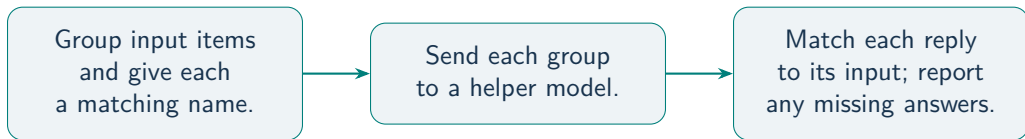
Statements per request	Correct judgments	Total time
48, without matching names	49%	19 seconds
48, with matching names	85%	91 seconds
16, without matching names	81%	19 seconds
1, without matching names	87%	31 seconds

Times cover all 768 statements, not one request. Sending one statement at a time was faster here, but repeated more input text.

One Qwen3-4B test on one A100; up to four requests ran at once, with input caching off. Times sum separate workload blocks, excluding startup. All 848 requests had valid formatting. Speed may differ on other setups. [E1]

Next test: let the RLM program organize helper requests and match their answers.

The main model would still choose what to ask. A new program component would handle groups of input items and their replies.



Compare fixed group sizes with the current RLM and with one item per request, using the same tasks, models, and computing budget.

Success means a more reliable **final calculation**, not just better helper answers.

Proposed component, not an established whole-RLM gain. Matching cannot guarantee correct reading, and identifiers are not a new invention. A later experiment could learn when to change group size.

We have a promising handoff result and a focused next research question.

What we know so far

Training examples helped the main model use helpers and calculate. Matching names improved the helpers' reading judgments without retraining.

What we want to establish next

Can the proposed program change improve the final answer at an acceptable cost in time and computation?

Would this be a worthwhile next study? What realistic task would make the test convincing?

The larger goal remains learning how to divide unfamiliar problems into useful smaller ones. Today's evidence is narrower. End of the main talk; the remaining pages are optional discussion material.

Example: the helper identifies question types, and Python adds the relevant points.

The points below are assigned test values, not confidence scores.

Add each user's points only for questions asking for a location. How many users have a total above 5?

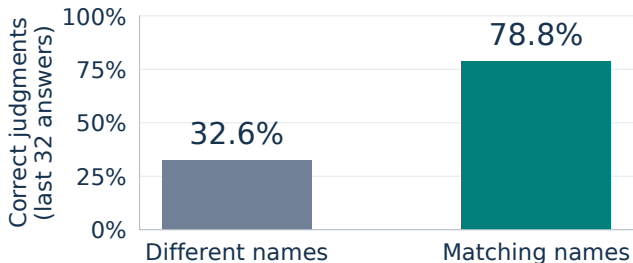
User	Question	Points	Asks for a location?
Ada	Where is Oslo?	4	Yes
Ada	What is the capital of Peru?	3	Yes
Ben	Who wrote Hamlet?	6	No
Ben	Where is Kyoto?	2	Yes

Ada: $4 + 3 = 7$. Ben: 2. Only Ada exceeds 5, so the answer is **1 user**.

Invented example, not a quoted training demonstration.

Using the same name on a statement and its answer helped more than using different names.

The helper judged statements about short texts. Answer order specified which statement to judge; both versions also supplied arbitrary names.



Input name → answer name

Different:

k7ab → z2pm

Matching:

k7ab → k7ab

Matching helped in all 16 input sets, even though both versions had valid formatting.

Qwen3-4B; only the last 32 judgments in each 48-statement request are scored here: 3,072 per bar. Different input sets from slide 5. This tests behavior, not the model's internal mechanism. [H6]

Reward training did not improve the final-answer count in this trial.

Reward training lets the model try solutions, then uses scores to adjust what it learns. We tested two reward rules from the same trained model.

Before reward training

57

out of 72 questions.

Correct-answer reward

55–57

out of 72 questions.

Answer reward plus agreement check

54

out of 72 questions.

The extra check rewarded agreement with Python's calculation from the helper replies. The numbers above count only correct final answers.

Each run attempted 576 practice solutions, then faced the same 72 test tasks. Two missing test results explain 55–57. Calculation review remains unfinished. One exploratory pair does not show that reward training generally fails. [R3]

Training also helped with new combinations of familiar steps.

Example of a new combination

Find users who asked for a location, such as “Where is Oslo?” Then count *their other questions* asking for a number, such as “How many moons does Mars have?”

Before worked-example training

2 / 72

correct answers and calculations.

After worked-example training

29 / 72

correct answers and calculations.

The instructions supplied these steps. Training helped the model carry out a new combination, not invent the plan.

One trained model, eight previously tested input sets. Missing outcomes allow 2–13 versus 29–30 successes. These combinations were absent from our demonstrations; this is not independent planning. [S4]

Misleading names can draw an answer toward the wrong input.

The helper judged statements about short texts, using misleading answer names.

Example of conflicting instructions

“Answer the statement in row 7, but put its answer beside the name row8.” Row 8 is a different input in the same request.

Name of another input,
such as row8

39%

of judgments were correct.

Unrelated name,
such as k7ab

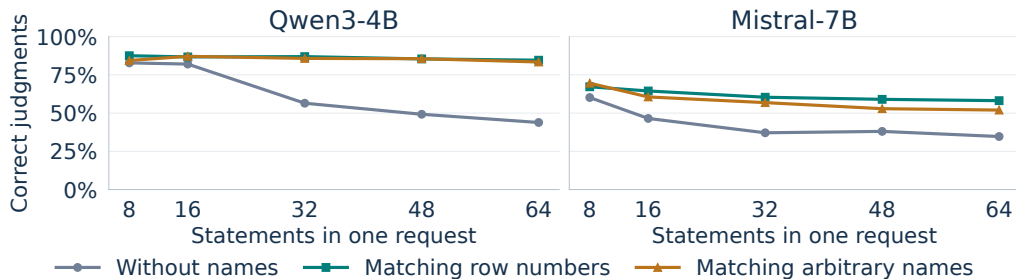
80%

of judgments were correct.

Invented illustration of a deliberate stress test. Software fixed names and order. Qwen3-4B; 16 groups of 48 statements. This measures output behavior, not attention. [H1]

Both row numbers and arbitrary names helped when helpers judged many statements together.

Two language models judged statements about short texts. We linked each statement to its answer using row numbers (1, 2, ...) or arbitrary names (k7ab, z2pm, ...).



Same 16 input sets; 240 requests per model. Three Mistral arbitrary-name responses had invalid formatting and count as wrong. This complicates comparing the two naming styles. Helper judgments, not complete RLM solutions. [H8, H9]